The tradeoff of enforcing backward-compatible changes is that we now have to treat as optional two fields that we need to have always filled in, and that may possibly fail validation if they're not. That is a common problem that can be solved over two subsequent releases of the application:

1. In the first release, you add the new columns as optional and implement a data migration to fill in the new columns for all the existing data. For Catalog Service, you could use a conventional value to represent that we don't know who created or updated the entity, such as `unknown` or `anonymous`.
2. In the second release, you can create a new migration to update the schema safely and make the new columns required.

I'll leave that up to you if you'd like to do that. If you're interested in implementing data migrations, I recommend that you check out Flyway's official documentation (https://flywaydb.org).

In the next section, you'll see how to test user-related auditing in Spring Data JDBC.

12.4.2 Testing data auditing with Spring Data and @WithMockUser

When we test security at the data layer, we're not interested in which authentication strategy has been adopted. The only thing we need is to know whether the operation is performed in the context of an authenticated request.

The Spring Security Test project provides us with a handy `@WithMockUser` annotation we can use on test cases to make them run in an authenticated context. You can also add information about the mock